

Metodologie di Programmazione (A-L)

Il semestre - a.a. 2025 – 2026

Design Pattern I

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

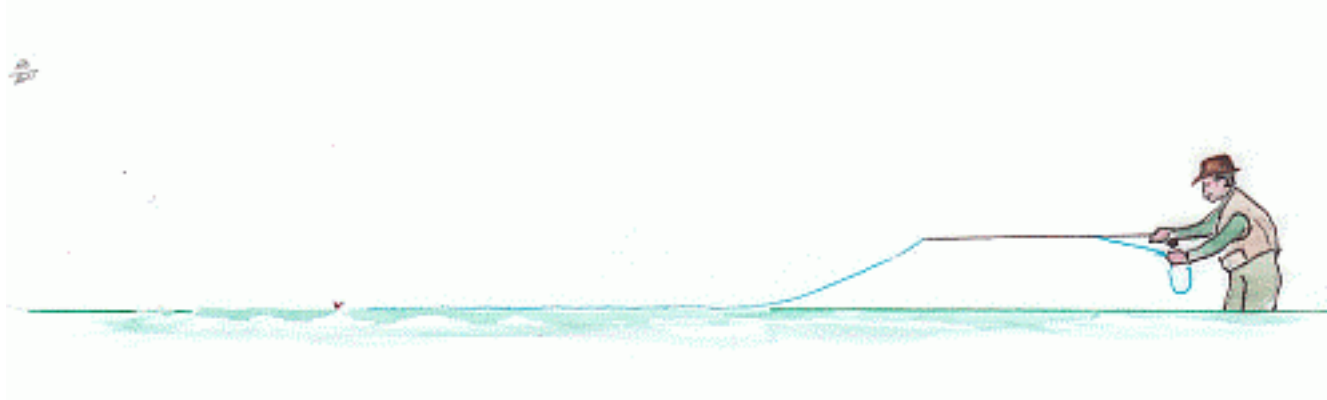
Design pattern parte I

Definizione

Un **Design Pattern** ("Modello di Progettazione"): è una **soluzione riutilizzabile** e collaudata ad un problema che si presenta comunemente in un contesto specifico durante la progettazione del software.

Definizione

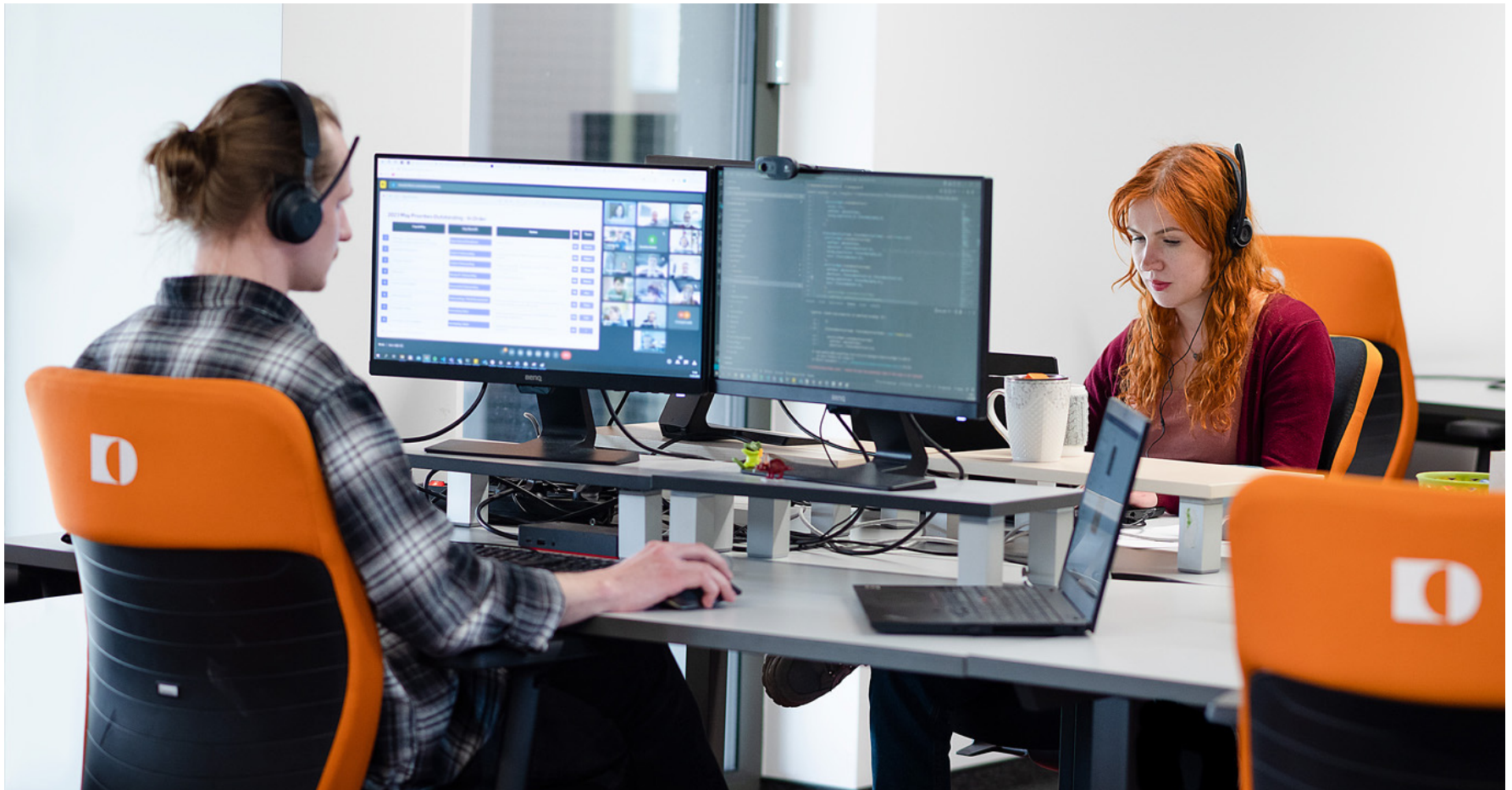
Non è codice finale: non è una libreria o un modulo software che puoi semplicemente copiare e incollare; è una **descrizione** o un **modello** su come risolvere un problema, che deve poi essere adattato al tuo contesto specifico.



È un po' come imparare l'arte di pescare...
Riguarda una **classe di problemi** e non riguarda solamente "catturare un pesce".

Definizione

È una soluzione collaudata: i design pattern non sono teorie astratte, ma **soluzioni** che sono state sviluppate e **perfezionate** nel tempo da **sviluppatrici/sviluppatori esperti**.



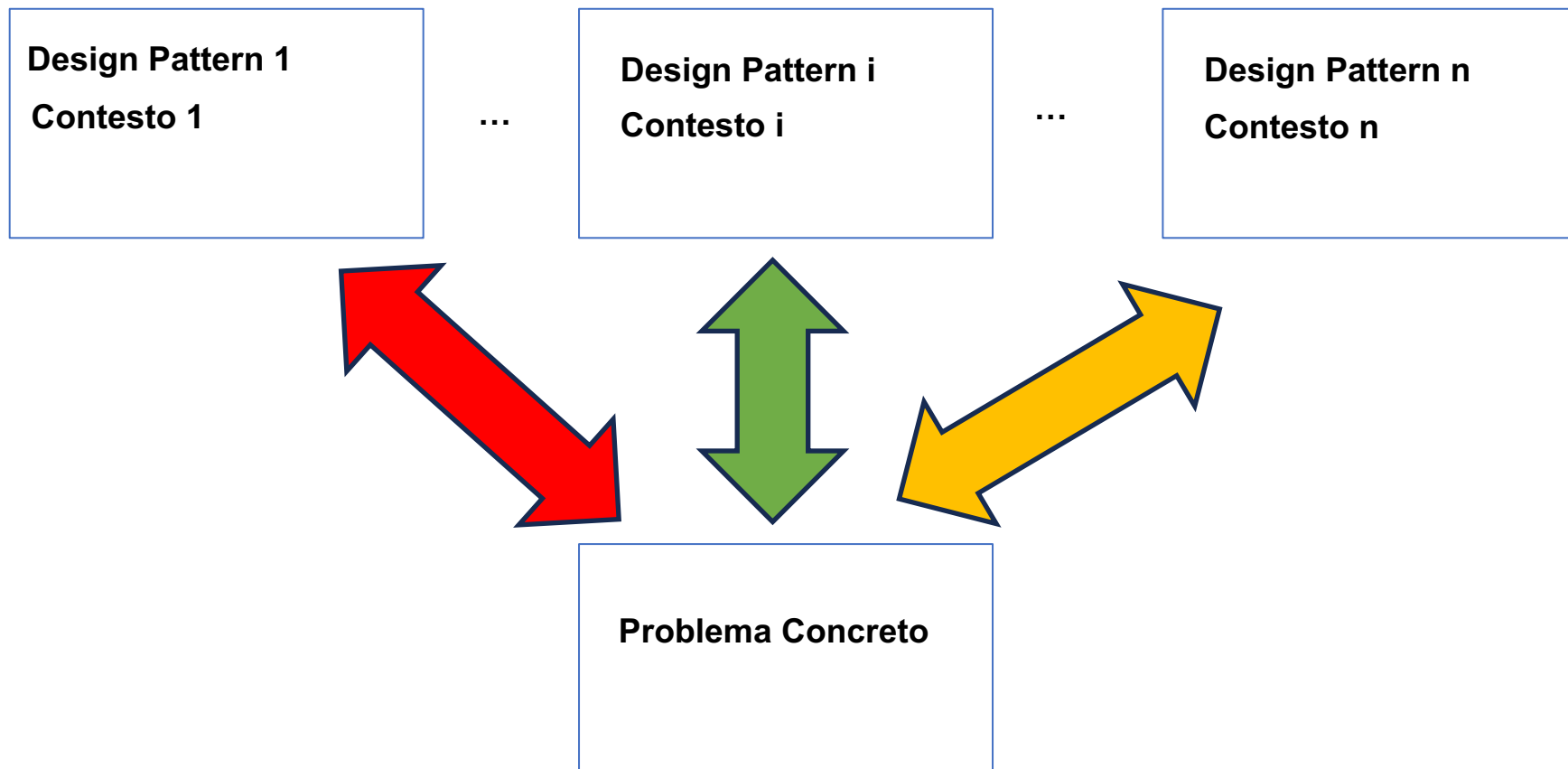
Definizione

Fornisce un vocabolario comune: Permette ai programmatori di comunicare in modo più efficiente sui problemi di progettazione. Ad esempio, dire *"Usiamo il pattern Factory Method qui"* trasmette immediatamente una quantità significativa di informazioni sulla struttura e sull'interazione degli oggetti.

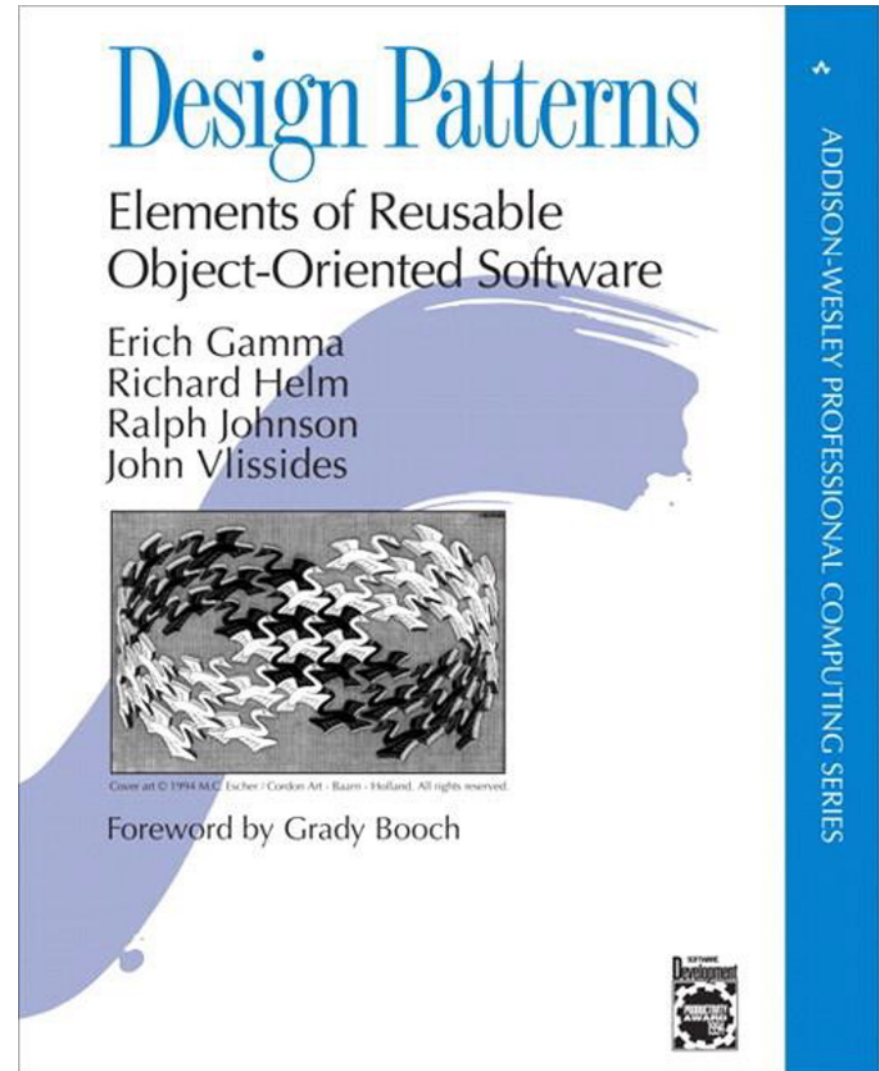
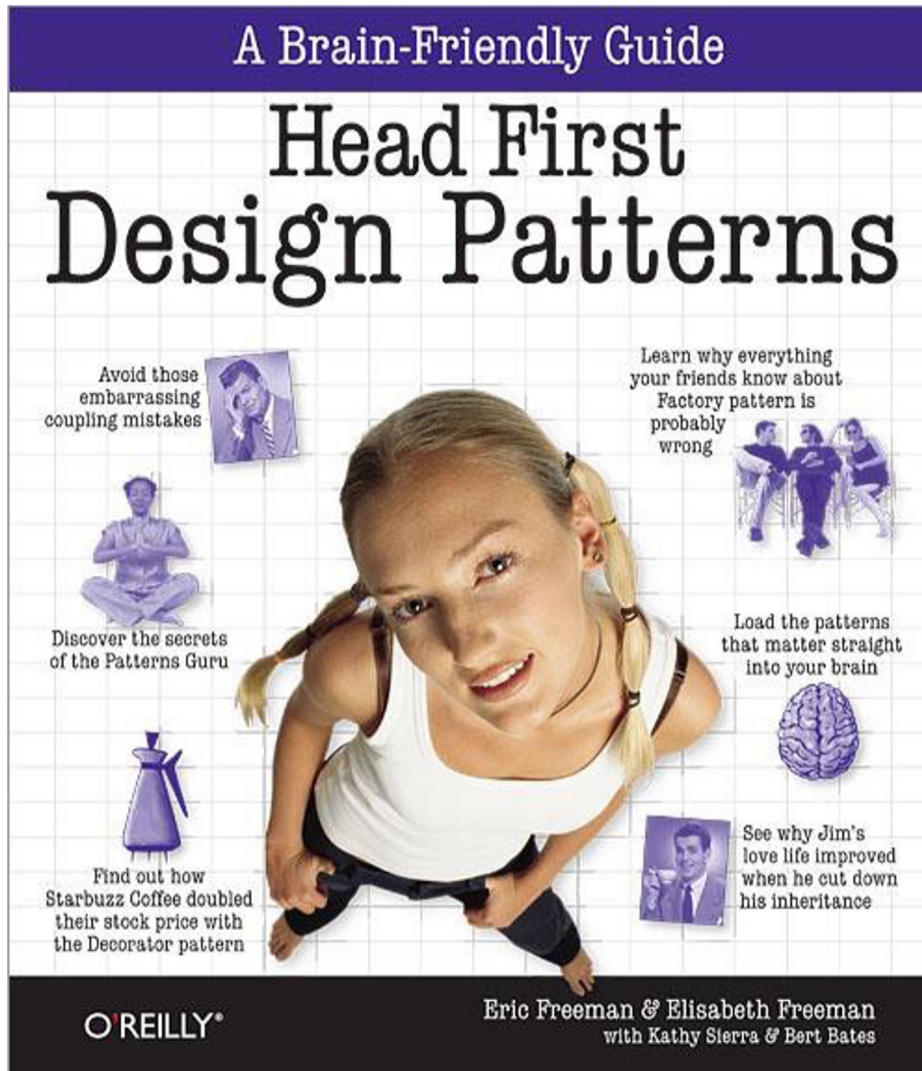


Definizione

Risponde a un contesto specifico: Ogni pattern descrive anche le circostanze (il *contesto*) in cui deve essere applicato e le conseguenze (i *compromessi*) della sua applicazione.



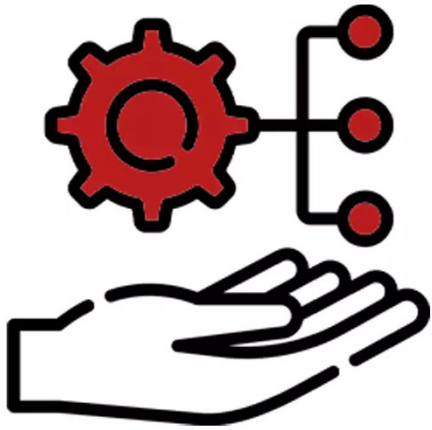
Per una trattazione in ampiezza e profondità



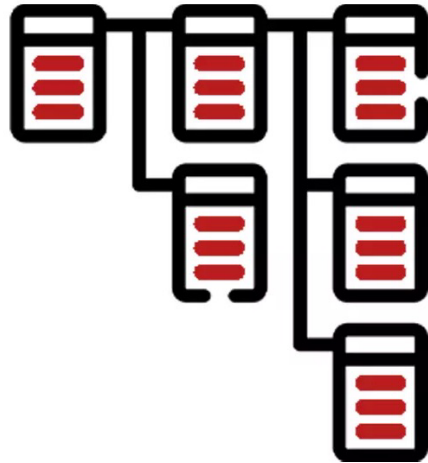
<https://design-patterns-101.vercel.app/>

Design Pattern: classificazione 1/4

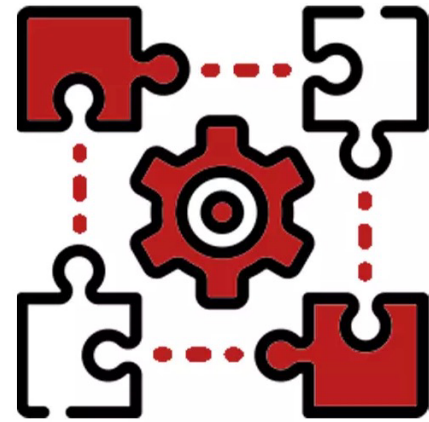
- 1) Pattern Creazionali (Creational Patterns)
- 2) Pattern Strutturali (Structural Patterns)
- 3) Pattern Comportamentali (Behavioral Patterns)



Creational Patterns



Structural Patterns

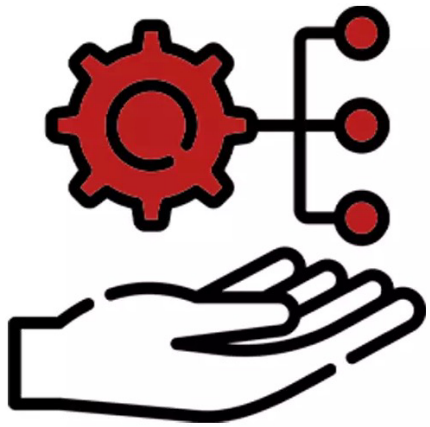


Behavioral Patterns

Design Pattern: classificazione 2/4

1. Pattern Creazionali (Creational Patterns)

Questi pattern riguardano i **processi di creazione degli oggetti**. Il loro scopo è disaccoppiare una classe dal modo in cui i suoi oggetti sono creati, composti e rappresentati. Essenzialmente, rendono un sistema più flessibile nel decidere quali oggetti devono essere creati per un dato scenario.



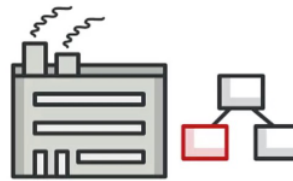
Creational Patterns

Creational



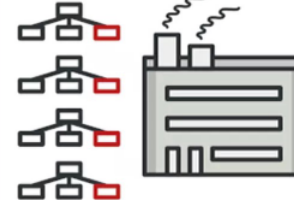
Singleton

Creational



Factory Method

Creational



Abstract Factory

Creational

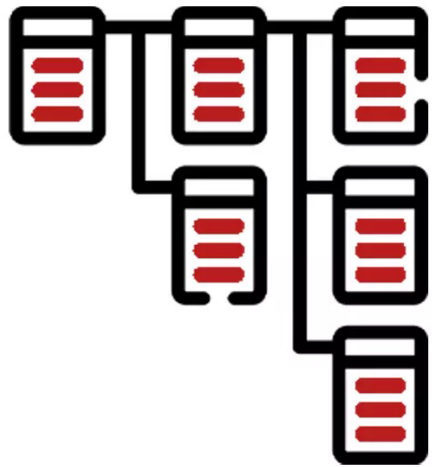


Builder

Design Pattern: classificazione 3/4

2. Pattern Strutturali (Structural Patterns)

Questi pattern si concentrano su come le classi e gli oggetti sono composti per formare strutture più grandi. Aiutano a garantire che, quando le parti di un sistema cambiano, l'intera struttura non abbia bisogno di essere modificata.



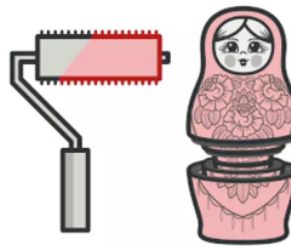
Structural Patterns

Structural



Facade

Structural

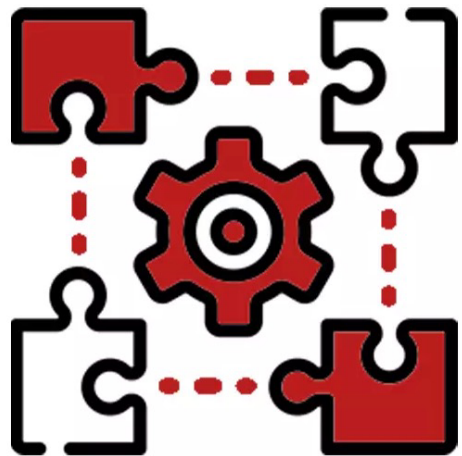


Decorator

Design Pattern: classificazione 4/4

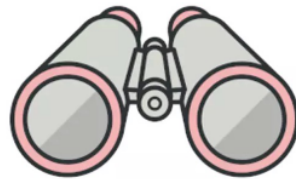
3. Pattern Comportamentali (Behavioral Patterns)

Questi pattern si occupano dell'interazione e della responsabilità tra gli oggetti. Descrivono modelli di comunicazione complessi e flussi di controllo tra diverse entità, rendendo il sistema più flessibile nei suoi schemi di interazione.



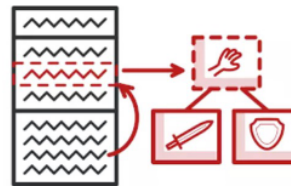
Behavioral Patterns

Behavioral

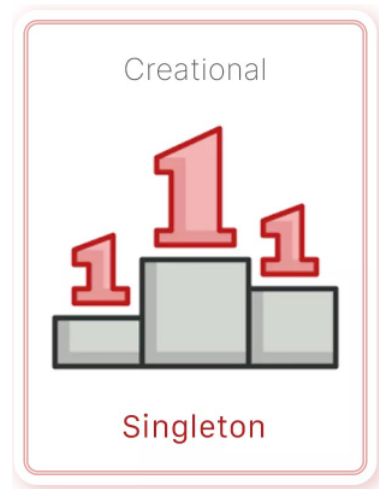


Observer

Behavioral



Strategy



Singleton pattern

Di ogni design pattern vedremo:

Introduzione

 Scopo

 Problema (contesto)

 Soluzione / Ricetta

 Esempio

Singleton pattern 1

- Singleton limita una classe ad avere una sola singola istanza (oggetto).
- Singleton fornisce un punto di accesso globale a questa istanza.
- Singleton è utile quando è necessario esattamente un oggetto per coordinare le azioni all'interno del sistema.



In termini pratici, il pattern **Singleton** garantisce che in tutta l'applicazione esista **una e una sola copia** di quell'oggetto e che tutti i componenti la utilizzino, impedendo la creazione di copie aggiuntive. È spesso usato per oggetti come registri, gestori di configurazione o pool di connessioni.

Singleton pattern 2

💬 Scopo

Il Singleton è un pattern di progettazione creazionale che ti permette di assicurare che una classe abbia una sola istanza, fornendo al contempo un punto di accesso globale a tale istanza.



😞 Problema

1) Assicurare che una classe abbia una sola istanza.

Perché qualcuno dovrebbe voler controllare quante istanze ha una classe?

La ragione più comune è per controllare l'accesso a qualche risorsa condivisa, per esempio, un database o un file.

2) Fornire un punto di accesso globale a tale istanza, senza utilizzare variabili globali (coding horror)

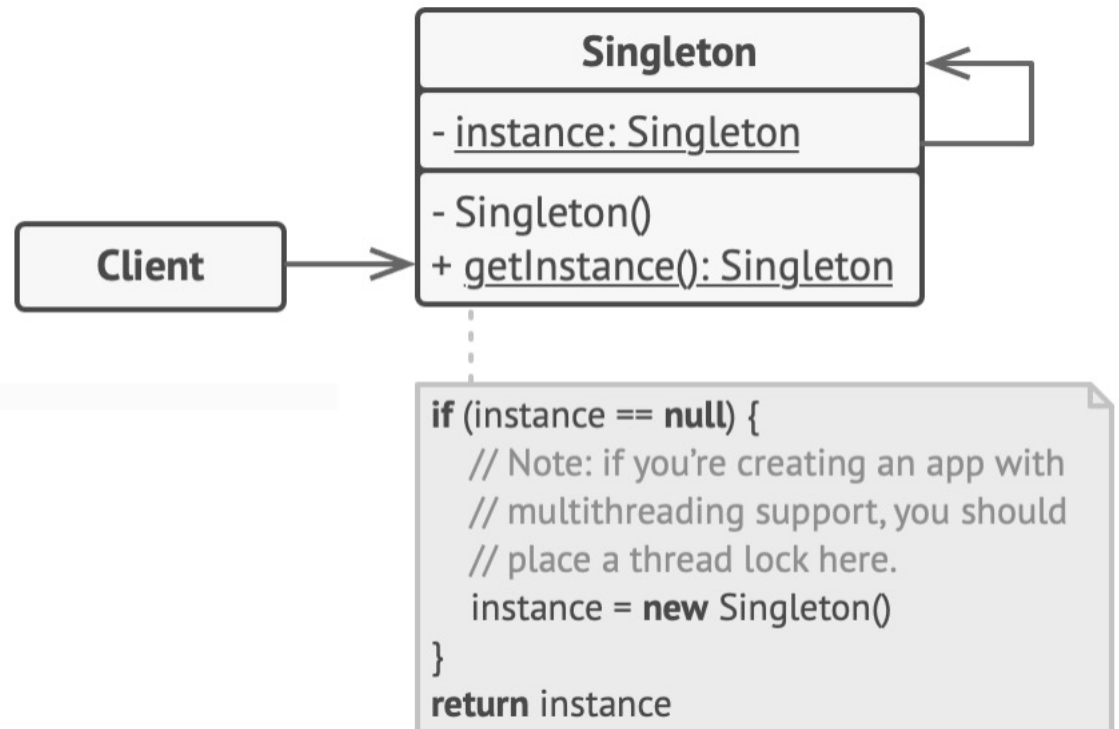
Ti ricordi di quelle variabili globali che tu usavi per memorizzare alcuni oggetti essenziali? Sebbene siano molto utili, sono anche molto insicure (**coding horror**) poiché chiunque può potenzialmente sovrascrivere il contenuto di quelle variabili e mandare in crash l'applicazione.

Singleton pattern 3

😊 Soluzione / Ricetta

1) La classe **Singleton** dichiara/incapsula il campo statico **instance** e il metodo statico **getInstance()** che restituisce la stessa istanza della sua propria classe.

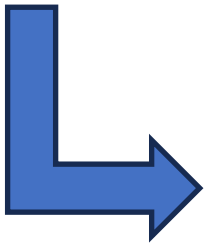
2) I costruttori del **Singleton** devono essere nascosti (privatizzandoli) al codice client. Chiamare il metodo **getInstance()** deve essere l'unico modo per ottenere l'oggetto **Singleton**.



Singleton pattern 4

Esempio

```
public class Classe {  
  
    public Classe() {  
        /* costruzione */  
    }  
  
}
```



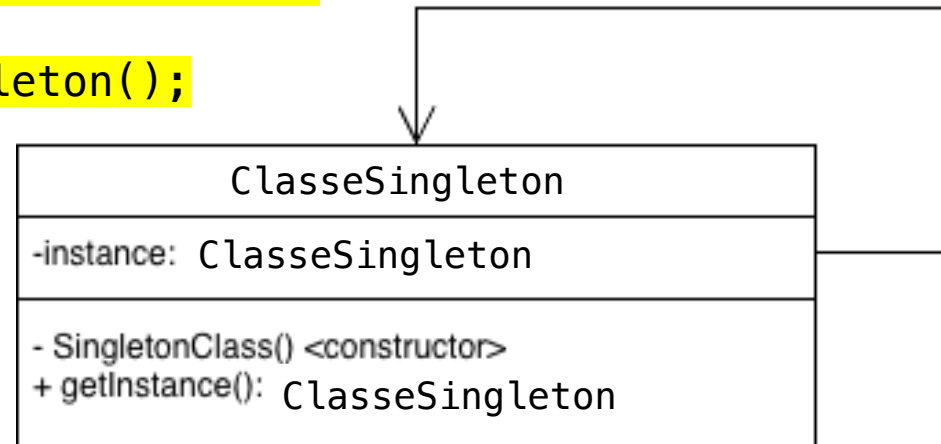
```
public class ClasseSingleton {  
  
    private static ClasseSingleton instance;  
  
    private ClasseSingleton() {  
        /* costruzione */  
    }  
  
    public static ClasseSingleton getInstance(){  
        if (instance==null)  
            instance=new ClasseSingleton();  
        return instance;  
    }  
}
```



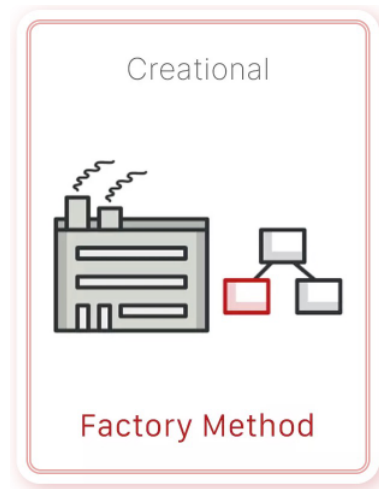
Singleton pattern 5

🚗 Esempio

```
public class ClasseSingleton {  
    private static ClasseSingleton instance;  
  
    private ClasseSingleton() {  
        /* costruzione */  
    }  
  
    public static ClasseSingleton getInstance(){  
        if (instance==null)  
            instance=new ClasseSingleton();  
        return instance;  
    }  
}
```



- 1) Non devo fare uso di variabili globali
- 2) Non si creano più istanze (in alcun modo)



Factory Method pattern

Factory Method pattern 1

Il Factory Method è un pattern di **progettazione creazionale** (come il Singleton) che si concentra sulla creazione di oggetti.

La sua funzione principale è:

Fornire **un'interfaccia** per creare oggetti in una superclasse, ma **permettere alle sottoclassi di decidere** quale classe esatta istanziare e restituire.



In termini più semplici, il Factory Method delega la responsabilità della creazione degli oggetti alle sottoclassi.

Factory Method pattern 2

💬 Scopo

Il Factory Method è un pattern di progettazione creazionale che fornisce un'interfaccia per la creazione di oggetti in una superclasse, ma permette alle sottoclassi di alterare il tipo di oggetti che verranno creati. **Aiuta a disaccoppiare il ruolo di creazione degli oggetti.**



😞 Problema

Abbiamo un'applicazione per la gestione della logistica.

La prima versione dell'app può gestire solo il trasporto tramite camion, quindi la maggior parte del tuo codice risiede all'interno della classe `Truck`.

Dopo un po', la tua app diventa piuttosto popolare. Ogni giorno ricevi decine di richieste da aziende di trasporto marittimo per incorporare la logistica via mare nell'app. Al momento, la maggior parte del tuo codice è accoppiata alla classe `Truck`. Aggiungere le Ships (Navi) all'app richiederebbe di apportare modifiche all'intero programma.

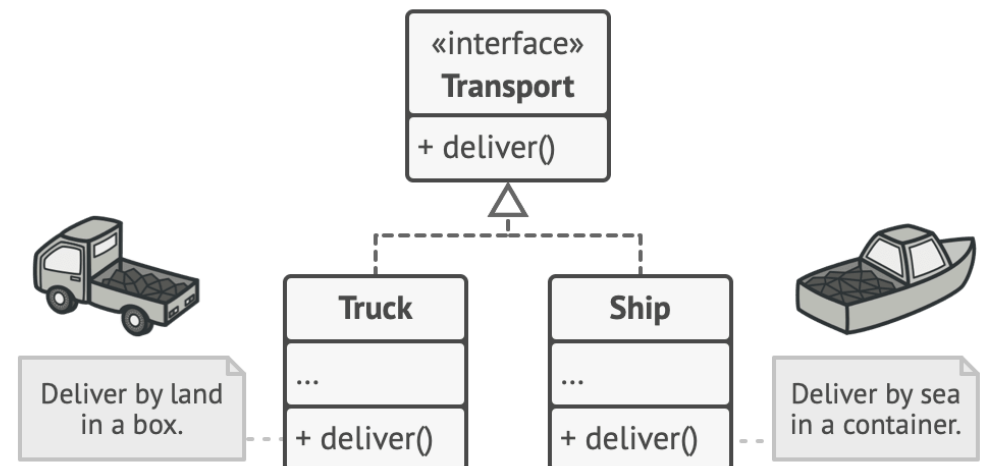
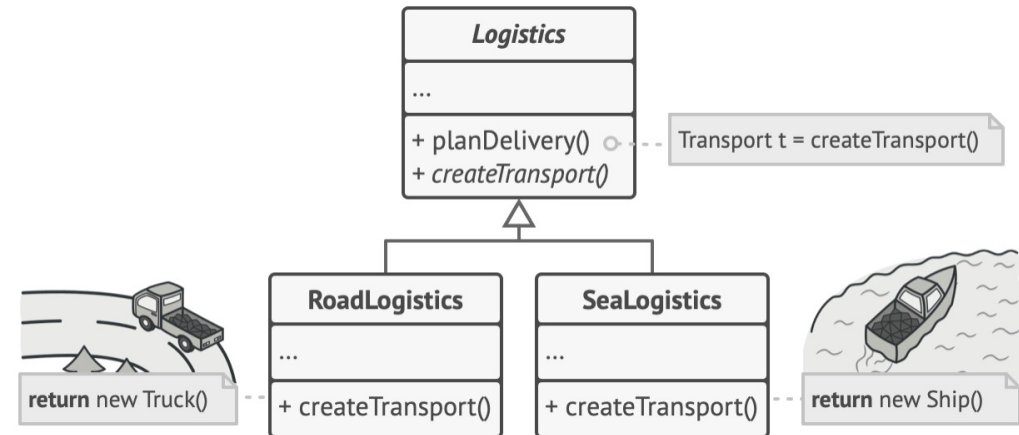
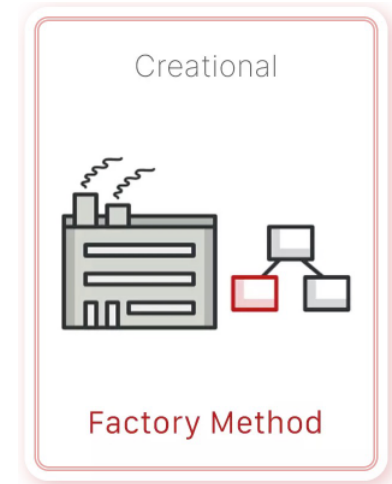
Inoltre, se in seguito decidessi di aggiungere un altro tipo di trasporto all'app, probabilmente dovresti apportare nuovamente tutte queste modifiche.

Di conseguenza, ti ritroverai con un codice piuttosto sgradevole, pieno di condizionali che modificano il comportamento dell'app a seconda della classe degli oggetti di trasporto.

Factory Method pattern 3

😊 Soluzione / Ricetta

- 1) Definire una astrazione della propria Classe.
- 2) Definire una interfaccia di Prodotto.
- 3) Dotare la Classe di un **metodo di fabbricazione** che ritornerà un Prodotto.
- 4) Fornire diverse implementazioni di Prodotto.
- 5) Specializzare il **metodo di fabbricazione** nelle proprie concretizzazioni di Classe.



Factory Method pattern 4

Esempio

```
public abstract class ClasseFactoryMethod {  
    abstract Prodotto getProdotto();  
}
```

```
public class Pastificio extends ClasseFactoryMethod{  
    @Override  
    Prodotto getProdotto() {  
        return new Pasta();  
    }  
}
```

```
public class Pizzeria extends ClasseFactoryMethod{  
    @Override  
    Prodotto getProdotto() {  
        return new Pizza();  
    }  
}
```

```
public interface Prodotto {  
    /* */  
}
```

```
public class Pizza implements Prodotto{  
    /* */  
}
```

```
public class Pasta implements Prodotto{  
    /* */  
}
```

il «Factory Method»

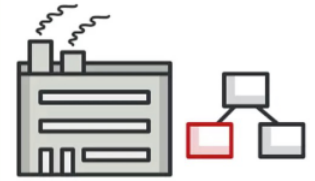


Factory Method pattern 5

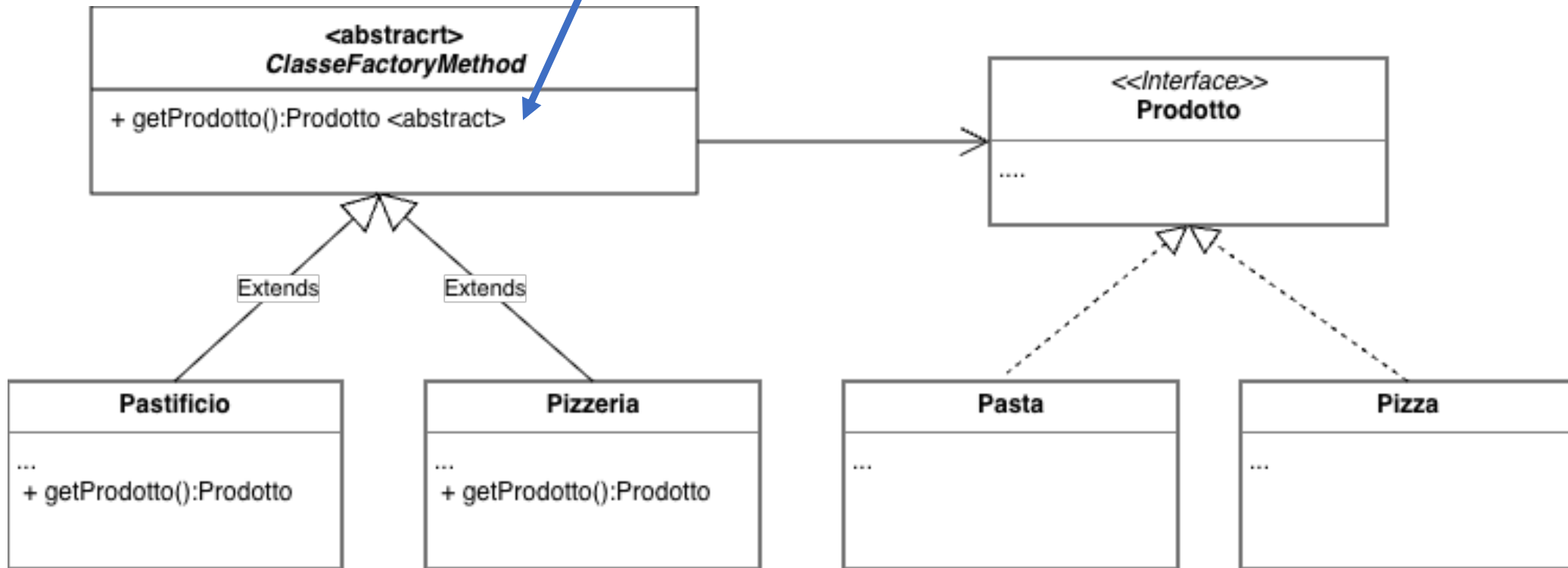
🚗 Esempio

il «Factory Method»

Creational



Factory Method



Posso aggiungere nel tempo:

- 1) Altri Fornitori
- 2) Altri Prodotti

Factory Method pattern 5

🚗 Esempio



Dobbiamo creare la nostra interfaccia Dialog che ha un bottone.

Ma non sappiamo su quale sistema operativo dovrà essere creata e quali sistemi operativi dovranno essere supportati.

```
abstract class Dialog {  
    public abstract Button createButton();  
  
    public void render() {  
        Button b = createButton();  
        b.draw();  
    }  
}
```


Factory Method pattern 5

Esempio



Dobbiamo creare la nostra interfaccia Dialog che ha un bottone.

Ma non sappiamo su quale sistema operativo dovrà essere creata e quali sistemi operativi dovranno essere supportati.

```
abstract class Dialog {  
  public abstract Button createButton();  
  
  public void render() {  
    Button b = createButton();  
    b.draw();  
  }  
}
```

Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli e <https://design-patterns-101.vercel.app/>

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM2011O000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.